

TABLE DES MATIÈRES

PARTIE I	– PHILOSOPHIE DE L'AUTOMATISATION EN DATA SCIENCE
PARTIE II	– LES PIPELINES DE DONNÉES (ETL / ELT)
PARTIE III	– AUTOMATISATION DU MACHINE LEARNING (AutoML & MLOps)
PARTIE IV	– ORCHESTRATION ET SCHEDULING
PARTIE V	– APIS ET MISE EN PRODUCTION DES MODÈLES
PARTIE VI	– MONITORING ET MAINTENANCE AUTOMATISÉE
PARTIE VII	– ROBOTIC PROCESS AUTOMATION (RPA) ET DATA SCIENCE
PARTIE VIII	– AGENTS IA ET AUTOMATISATION COGNITIVE
PARTIE IX	– SÉCURITÉ, GOUVERNANCE ET ÉTHIQUE DE L'AUTOMATISATION
PARTIE X	– CAS PRATIQUES ET ARCHITECTURES RÉELLES

PARTIE I – PHILOSOPHIE DE L'AUTOMATISATION EN DATA SCIENCE

=====

1.1 POURQUOI AUTOMATISER – LA VÉRITÉ QUE LES JUNIORS IGNORENT

=====

DÉFINITION OPÉRATIONNELLE :

"L'automatisation en data science est la capacité à transformer un processus manuel, répétitif et humain-dépendant en un système autonome, fiable, traçable et scalable qui produit de la valeur sans intervention constante."

L'ERREUR CLASSIQUE DU JUNIOR :

Il croit que son travail est de construire des modèles.
Le senior sait que son vrai travail est de construire des SYSTÈMES qui construisent, évaluent et déploient des modèles – automatiquement.

Un modèle sans automatisation = un prototype.

Un modèle avec automatisation = un produit.

LES TROIS NIVEAUX D'AUTOMATISATION

NIVEAU 1 – AUTOMATISATION DES TÂCHES

- Scripts Python planifiés, rapports automatiques, alertes
- Gain : éliminer le travail répétitif manuel
- Compétence clé : scripting, cron jobs, scheduling

NIVEAU 2 – AUTOMATISATION DES PIPELINES

- ETL/ELT automatisés, pipelines ML, feature stores
- Gain : données toujours fraîches, modèles toujours à jour
- Compétence clé : Airflow, dbt, MLflow, CI/CD

NIVEAU 3 – AUTOMATISATION COGNITIVE (Agents IA)

- Systèmes qui raisonnent, décident et agissent de façon autonome
- Gain : remplacer des processus complexes multi-étapes
- Compétence clé : LLMs, agents, RAG, orchestration d'agents

PRINCIPE DE LA PYRAMIDE D'AUTOMATISATION

AGENTS IA AUTONOMES	← Niveau 3 (cognitif)
PIPELINES ML / MLOPS	← Niveau 2 (systémique)
SCRIPTS & SCHEDULING	← Niveau 1 (opérationnel)

Chaque niveau repose sur le précédent.

On ne bâtit pas d'agents IA sans pipelines solides.

On ne bâtit pas de pipelines sans scripts fiables.

[P1] IDEMPOTENCE

- Exécuter un pipeline deux fois doit produire le même résultat qu'une fois.
- Pas d'effets de bord cumulatifs.
- Clé pour la reprise sur erreur.

[P2] OBSERVABILITÉ

- Chaque étape doit être visible, mesurable et traçable.
- Logs structurés, métriques, alertes.
- "Ce qu'on ne mesure pas, on ne peut pas l'améliorer."

[P3] MODULARITÉ

- Découper en unités indépendantes, testables séparément.
- Chaque transformation = une fonction pure.
- Facilite la maintenance et le débogage.

[P4] RÉILIENCE

- Le système doit gérer les erreurs gracieusement.
- Retry automatique, dead letter queues, fallbacks.
- Un pipeline qui crash silencieusement est pire que l'absence de pipeline.

[P5] VERSIONNEMENT

- Code, données, modèles, configurations → tout versionner.
- Git pour le code, DVC pour les données, MLflow pour les modèles.
- Reproductibilité garantie dans le temps.

[P6] DOCUMENTATION AUTOMATIQUE

- Les systèmes automatisés doivent se documenter eux-mêmes.
- Métadonnées sur chaque exécution, schéma des données, lineage.

PARTIE II – LES PIPELINES DE DONNÉES (ETL / ELT)

2.1 ETL VS ELT – COMPRENDRE L'ARCHITECTURE

ETL (Extract → Transform → Load)

Paradigme classique :

- [1] EXTRACT : extraire les données depuis les sources (DB, API, fichiers)
- [2] TRANSFORM : nettoyer, agréger, enrichir avant chargement
- [3] LOAD : charger dans l'entrepôt cible (Data Warehouse)

Usage : volumes modérés, transformation complexe, data warehouses classiques

Outils : Talend, Informatica, SSIS, Apache Spark, Python custom

ELT (Extract → Load → Transform)

Paradigme moderne :

- [1] EXTRACT : extraire les données brutes
- [2] LOAD : charger immédiatement dans le data lake / warehouse
- [3] TRANSFORM : transformer dans le warehouse avec SQL ou dbt

Usage : volumes massifs, cloud data warehouses (BigQuery, Snowflake, Redshift)

Avantage : conserver les données brutes, transformer à la demande

Outils : dbt, Fivetran, Airbyte, dlt (data load tool)

ARCHITECTURE MEDALLION (Bronze / Silver / Gold)

BRONZE (Raw Layer)

- Données brutes, non transformées, telles qu'elles arrivent
- Immuables – jamais modifier les données bronze
- Schéma : source_table_YYYYMMDD

SILVER (Cleaned Layer)

- Données nettoyées, déduplicées, standardisées
- Jointures entre sources réalisées ici
- Schéma unifié et validé

GOLD (Business Layer)

- Agrégations métier, KPIs, tables prêtes pour les dashboards et ML
- Optimisées pour la lecture (dénormalisées)

→ Directement consommées par les applications et modèles

FLUX :

Sources → [BRONZE] → [SILVER] → [GOLD] → Dashboards / ML / APIs

2.2 CONSTRUCTION D'UN PIPELINE DE DONNÉES AUTOMATISÉ

ANATOMIE D'UN PIPELINE PROFESSIONNEL

- [1] INGESTION (Source → Stockage brut)
 - Connecteurs vers : bases SQL, APIs REST, fichiers S3/GCS, Kafka, webhooks
 - Modes : batch (quotidien, horaire) ou streaming (temps réel)
 - Gestion : schéma évolutif, backfill, deduplication à l'ingestion

- [2] VALIDATION DES DONNÉES (Data Quality)
 - Vérifier avant de transformer :
 - Complétude : taux de valeurs manquantes < seuil
 - Unicité : pas de doublons sur les clés primaires
 - Fraîcheur : les données ont bien été mises à jour
 - Plausibilité : valeurs dans les bornes attendues
 - Outils : Great Expectations, dbt tests, Soda Core, Pandera

EXEMPLE DE TEST GREAT EXPECTATIONS :

```
expect_column_values_to_not_be_null("user_id")
expect_column_values_to_be_between("age", 0, 120)
expect_column_unique_value_count_to_be_between("country", 1, 200)
```

- [3] TRANSFORMATION (nettoyage, enrichissement, agrégation)
 - Nettoyer : imputation, standardisation, déduplication
 - Enrichir : jointures avec tables de référence, géocodage, scoring
 - Agréger : métriques par période, par segment, par entité
- [4] CHARGEMENT (Destination)
 - Full refresh : réécrire toute la table à chaque run
 - Incremental : n'ajouter que les nouvelles lignes (delta)
 - Upsert : insérer ou mettre à jour si la clé existe déjà
 - Choisir selon la taille et la fréquence
- [5] NOTIFICATION ET RAPPORT
 - Alerter en cas d'échec (Slack, email, PagerDuty)
 - Rapport de qualité automatique à chaque exécution
 - Logging structuré (JSON) pour analyse ultérieure

PIPELINE PYTHON – STRUCTURE TYPE

```
class DataPipeline:

    def extract(self, source: str) -> pd.DataFrame:
        """Extraire les données depuis la source."""
        logger.info(f"Extraction depuis {source}")
        try:
            df = pd.read_sql(query, connection)
            logger.info(f"{len(df)} lignes extraites")
            return df
        except Exception as e:
            logger.error(f"Erreur extraction : {e}")
            raise

    def validate(self, df: pd.DataFrame) -> bool:
        """Valider la qualité des données."""
        assert df['id'].nunique() == len(df), "Doublons détectés"
        assert df['montant'].between(0, 1e9).all(), "Valeurs hors bornes"
        assert df['date'].notna().mean() > 0.99, "Trop de dates manquantes"
        return True

    def transform(self, df: pd.DataFrame) -> pd.DataFrame:
        """Transformer et enrichir les données."""
        df['montant_eur'] = df['montant'] * get_exchange_rate()
        df['segment'] = df['montant_eur'].apply(classify_segment)
        df['date'] = pd.to_datetime(df['date'])
        return df

    def load(self, df: pd.DataFrame, destination: str) -> None:
```

```

        """Charger les données dans la destination."""
        df.to_sql(destination, engine, if_exists='append', index=False)
        logger.info(f"{len(df)} lignes chargées dans {destination}")

    def run(self):
        """Exécuter le pipeline complet avec gestion d'erreurs."""
        df = self.extract(SOURCE)
        if self.validate(df):
            df_clean = self.transform(df)
            self.load(df_clean, DESTINATION)

```

2.3 DBT – LA RÉVOLUTION DE LA TRANSFORMATION AUTOMATISÉE

dbt (data build tool) = framework de transformation SQL avec versionnement, tests automatiques, documentation et lineage intégrés.

PHILOSOPHIE dbt :

"Les données transformées = du code SQL. Traitons-le comme du code."
 → Git pour versionner, tests pour valider, CI/CD pour déployer.

STRUCTURE D'UN PROJET dbt

```

models/
  staging/      ← nettoyage basique des sources (Bronze → Silver)
  intermediate/ ← logique métier complexe
  marts/        ← tables Gold exposées aux utilisateurs finaux
tests/          ← tests personnalisés
macros/         ← fonctions SQL réutilisables
seeds/          ← données de référence statiques (CSV)
snapshots/      ← historisation des changements (SCD Type 2)

```

MODÈLE dbt EXEMPLE (models/marts/ventes_mensuelles.sql)

```

{{ config(materialized='table') }}

WITH ventes AS (
    SELECT * FROM {{ ref('stg_commandes') }}
    WHERE statut = 'validé'
),
aggregation AS (
    SELECT
        DATE_TRUNC('month', date_commande) AS mois,
        region,
        SUM(montant_eur) AS ca_total,
        COUNT(*) AS nb_commandes,
        COUNT(DISTINCT client_id) AS nb_clients_uniques
    FROM ventes
    GROUP BY 1, 2
)
SELECT * FROM aggregation

```

TESTS dbt AUTOMATIQUES (schema.yml)

```

models:
  - name: ventes_mensuelles
    columns:
      - name: mois
        tests:
          - not_null
          - unique
      - name: ca_total
        tests:
          - not_null
          - dbt_utils.accepted_range:
              min_value: 0

```

→ dbt test : exécute tous les tests
 → dbt docs generate && dbt docs serve : documentation interactive avec lineage

2.4 STREAMING ET TRAITEMENT EN TEMPS RÉEL

BATCH VS STREAMING

BATCH

- Traiter des volumes de données à intervalles fixes (heure, jour)
- Simple à implémenter et déboguer
- Latence : minutes à heures
- Cas d'usage : rapports quotidiens, réentraînement de modèles

STREAMING

- Traiter chaque événement dès son arrivée
- Complexité accrue (état distribué, exactly-once)
- Latence : millisecondes à secondes
- Cas d'usage : détection de fraude en temps réel, alertes IoT

KAFKA – COLONNE VERTÉBRALE DU STREAMING

Architecture producteur → topic → consommateur

- Topic : canal nommé où les messages sont publiés
- Partition: subdivision du topic pour la parallélisation
- Offset : position d'un message dans une partition (immuable)
- Consumer Group : ensemble de consommateurs coopérant

Garanties : durabilité (réplication), ordre dans une partition, rétention configurable (7 jours par défaut).

APACHE FLINK / SPARK STREAMING

- Flink : streaming natif, faible latence, état distribué, exactement-once
- Spark Streaming : micro-batches, riche écosystème, plus simple à opérer
- Kafka Streams : bibliothèque Java légère pour traitement dans Kafka

OPÉRATIONS CLÉS SUR UN STREAM :

- map() : transformer chaque événement
- filter() : garder uniquement certains événements
- window() : agréger sur une fenêtre temporelle
- join() : joindre deux streams en temps réel
- aggregate() : calculer des métriques glissantes

FENÊTRES TEMPORELLES :

- Tumbling window : fenêtres fixes non-chevauchantes (ex: 1 min par 1 min)
- Sliding window : fenêtres chevauchantes (ex: 1 min toutes les 30 sec)
- Session window : fenêtres basées sur l'activité (inactivité = nouvelle fenêtre)

PARTIE III – AUTOMATISATION DU MACHINE LEARNING (AutoML & MLOps)

3.1 MLOPS – L'INGÉNIERIE DU ML EN PRODUCTION

DÉFINITION :

MLOps = DevOps appliqué au Machine Learning.
L'art de construire des systèmes qui entraînent, évaluent, déploient et surveillent des modèles ML de façon automatisée et fiable.

PROBLÈME QUE MLOPS RÉSOUT :

"Le gouffre de déploiement" – 87% des projets ML ne passent jamais en prod.
Raisons : reproductibilité impossible, maintenance difficile, monitoring absent, collaboration data scientist / ingénieur inexistante.

MLOps comble ce gouffre.

LES COMPOSANTES DU MLOPS

[1] VERSIONNEMENT

- Code : Git (GitHub, GitLab)
- Données : DVC, Delta Lake, LakeFS
- Modèles : MLflow Model Registry, W&B Artifacts
- Config : Hydra, YAML versionné

[2] EXPÉRIMENTATION TRAÇABLE

- Chaque run d'entraînement logué automatiquement : paramètres, métriques, artefacts, environnement
- Comparaison d'expériences en un coup d'œil
- Outils : MLflow Tracking, Weights & Biases, Neptune.ai, Comet

EXEMPLE MLFLOW :

```
import mlflow
```

```

with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.01)
    mlflow.log_param("n_estimators", 100)
    model = train_model(X_train, y_train, lr=0.01, n=100)
    acc = evaluate(model, X_test, y_test)
    mlflow.log_metric("accuracy", acc)
    mlflow.sklearn.log_model(model, "model")

```

[3] PIPELINES D'ENTRAÎNEMENT AUTOMATISÉS

- Déclenché par : nouveau commit, nouvelles données, schedule
- Étapes : load data → validate → feature engineering → train → evaluate
- Si métriques satisfaisantes → enregistrer dans le registry

[4] MODEL REGISTRY

- Catalogue centralisé des modèles avec leurs versions
- Étapes de cycle de vie : Staging → Production → Archived
- Promotion manuelle ou automatique selon seuils de performance

[5] DÉPLOIEMENT AUTOMATISÉ (CD – Continuous Delivery)

- Modèle approuvé → déploiement automatique en prod
- Stratégies : canary, blue-green, shadow mode
- Rollback automatique si dégradation détectée

[6] MONITORING EN PRODUCTION

- Surveiller les performances du modèle dans le temps
- Détecter le drift des données et des prédictions
- Déclencher le réentraînement si nécessaire

3.2 CI/CD POUR LE MACHINE LEARNING

CI (Continuous Integration)

À chaque push de code :

- [1] Lint et formatage (flake8, black, isort)
- [2] Tests unitaires du code de traitement
- [3] Tests de la qualité des données (Great Expectations)
- [4] Tests du modèle (performance sur un dataset de référence)
- [5] Build de l'image Docker du modèle

CD (Continuous Delivery/Deployment)

Si CI passe :

- [1] Entraîner le modèle sur les données complètes
- [2] Évaluer vs modèle en production (champion vs challenger)
- [3] Si nouveau modèle > ancien → promotion automatique
- [4] Déploiement en staging → tests d'intégration
- [5] Déploiement en production → monitoring activé

EXEMPLE PIPELINE GITHUB ACTIONS (.github/workflows/ml_pipeline.yml)

```

name: ML Pipeline
on:
  push:
    branches: [main]
  schedule:
    - cron: '0 2 * * 1' # Chaque lundi à 2h du matin

jobs:
  train_and_evaluate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run data validation
        run: python scripts/validate_data.py
      - name: Train model
        run: python scripts/train.py
      - name: Evaluate model
        run: python scripts/evaluate.py

```

```
- name: Deploy if better
run: python scripts/deploy_if_better.py
```

3.3 AUTOML – AUTOMATISER LA CONSTRUCTION DES MODÈLES

DÉFINITION :

AutoML automatise le processus de sélection d'algorithmes, feature engineering, et optimisation des hyperparamètres – des tâches chronophages pour les data scientists.

NIVEAUX D'AUTOMATISATION EN AUTOML

[1] HYPERPARAMETER OPTIMIZATION (HPO)

Automatiser la recherche des meilleurs hyperparamètres.

Grid Search : essayer toutes les combinaisons (exponentiellement coûteux)
Random Search: échantillonner aléatoirement (meilleur que grid pour ≥ 3 params)
Bayesian Opt : utiliser les résultats précédents pour guider la recherche
→ Optuna, Hyperopt, Ray Tune

EXEMPLE OPTUNA :

```
import optuna

def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)
    n_layers = trial.suggest_int("n_layers", 1, 5)
    model = build_model(lr, n_layers)
    return cross_val_score(model, X, y).mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=100, n_jobs=-1)
best_params = study.best_params
```

[2] NEURAL ARCHITECTURE SEARCH (NAS)

Automatiser la conception des architectures de réseaux de neurones.
→ EfficientNet, DARTS, ProxylessNAS
→ Très coûteux en calcul → utilisé par les grandes entreprises

[3] AUTOML END-TO-END

Pipeline complet automatisé : features → algo → hyperparams → évaluation

- Auto-sklearn : meta-learning + Bayesian optimization (sklearn)
- TPOT : algorithmes évolutionnaires sur les pipelines sklearn
- H2O AutoML : production-ready, supporte classification/régression/time series
- Google AutoML : service cloud, nécessite peu de compétences techniques
- AutoGluon : Amazon, ensembling automatique, très performant

PIÈGE AUTOML :

AutoML accélère l'expérimentation mais ne remplace pas la compréhension métier et la préparation des données. Un AutoML sur des données mal préparées produira le meilleur modèle d'un mauvais problème.

3.4 FEATURE STORE – AUTOMATISER LA PRODUCTION DES FEATURES

PROBLÈME SANS FEATURE STORE :

- Chaque data scientist recalcule les mêmes features
- Incohérence entre features d'entraînement et de production
- Duplication de code, erreurs de leakage temporel

FEATURE STORE = RÉFÉRENTIEL CENTRALISÉ DES FEATURES

Composantes :

- Feature Registry : catalogue des features avec métadonnées
- Offline Store : features historiques pour l'entraînement (ex: Parquet/S3)
- Online Store : features en temps réel pour l'inférence (ex: Redis, DynamoDB)
- Feature Pipeline : calcul automatique et mise à jour des features
- Point-in-time join : join correct en respectant la chronologie (évite le leakage)

Outils : Feast (open source), Tecton, Hopsworx, Vertex AI Feature Store

EXEMPLE FEAST :

```
from feast import FeatureStore
```

```

store = FeatureStore(repo_path="feature_repo/")

# Récupérer des features pour l'entraînement (offline)
training_df = store.get_historical_features(
    entity_df=entity_df,
    features=["user_stats:nb_commandes_30j", "user_stats:ca_30j"]
).to_df()

# Récupérer des features pour l'inférence (online, temps réel)
feature_vector = store.get_online_features(
    features=["user_stats:nb_commandes_30j"],
    entity_rows=[{"user_id": 123}]
).to_dict()

```

PARTIE IV – ORCHESTRATION ET SCHEDULING

4.1 APACHE AIRFLOW – L'ORCHESTRATEUR DE RÉFÉRENCE

DÉFINITION :

Airflow est une plateforme d'orchestration de workflows définis en Python sous forme de DAGs (Directed Acyclic Graphs).

DAG = graphe orienté sans cycle = séquence de tâches avec dépendances.

CONCEPTS FONDAMENTAUX

DAG (Directed Acyclic Graph)

- Définit le workflow : quelles tâches, dans quel ordre, avec quelles dépendances
- Planifié par un schedule (cron expression ou preset)
- Chaque DAG a un dag_id unique

Task (Opérateur)

- Unité de travail dans un DAG
- Types : PythonOperator, BashOperator, SQLOperator, EmailOperator, etc.
- Chaque tâche a un task_id unique dans le DAG

DAG Run

- Une exécution concrète d'un DAG à une date donnée
- État : running, success, failed, skipped

XCom (Cross-communication)

- Mécanisme de partage de données légères entre tâches

EXEMPLE DAG COMPLET (Pipeline ML automatisé)

```

from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.email import EmailOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data-team',
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
    'email_on_failure': True,
    'email': ['alertes@company.com'],
}

with DAG(
    dag_id='pipeline_ml_ventes',
    default_args=default_args,
    schedule_interval='0 6 * * 1', # Chaque lundi à 6h
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=['ml', 'ventes'],
) as dag:

    t1_extract = PythonOperator(
        task_id='extraire_donnees',
        python_callable=extract_sales_data,

```



```

)

t2_validate = PythonOperator(
    task_id='valider_qualite',
    python_callable=validate_data_quality,
)

t3_features = PythonOperator(
    task_id='calculer_features',
    python_callable=compute_features,
)

t4_train = PythonOperator(
    task_id='entraîner_model',
    python_callable=train_model,
)

t5_evaluate = PythonOperator(
    task_id='evaluer_model',
    python_callable=evaluate_and_register,
)

t6_notify = EmailOperator(
    task_id='notifier_equipe',
    to='team@company.com',
    subject='Pipeline ML terminé – {{ ds }}',
    html_content='Le modèle a été mis à jour avec succès.',
)

# Définir les dépendances
t1_extract >> t2_validate >> t3_features >> t4_train >> t5_evaluate >> t6_notify

```

BONNES PRATIQUES AIRFLOW

- Ne jamais faire de calcul lourd directement dans le DAG (imports globaux)
→ Mettre la logique dans des fonctions externes appelées par PythonOperator
- Utiliser les Variables Airflow pour les configs (URLs, chemins, seuils)
- Utiliser les Connections Airflow pour les credentials (jamais en dur)
- Paramétrer les DAGs avec {{ ds }}, {{ execution_date }}, {{ params }}
- Activer le DAG tagging pour filtrer dans l'UI
- Dimensionner les workers selon la charge (CeleryExecutor pour scale-out)

4.2 ALTERNATIVES À AIRFLOW

PREFECT

- Plus moderne et Pythonique qu'Airflow
- Gestion d'état distribuée, observabilité native
- Déploiement cloud ou self-hosted
- Code plus simple à écrire et tester

EXEMPLE PREFECT :

```

from prefect import flow, task

@task(retries=3, retry_delay_seconds=60)
def extract(): return pd.read_sql(query, conn)

@task
def transform(df): return df.dropna().assign(segment=classify)

@task
def load(df): df.to_sql("table", engine)

@flow(name="pipeline-ventes")
def pipeline():
    df = extract()
    df_clean = transform(df)
    load(df_clean)

pipeline.serve(cron="0 6 * * 1")

```

DAGSTER

- Orienté "assets" plutôt que tâches
- Lineage des données natif

- Tests et types stricts
- Idéal pour des équipes data engineering matures

LUIGI (Spotify)

- Plus ancien, plus simple
- Adapté aux pipelines batch simples sans UI riche

CRON (Unix)

- Le plus simple : planifier des scripts shell ou Python
- Pas d'UI, pas de retry automatique, pas de dépendances complexes
- Suffisant pour les pipelines simples à faible criticité

SYNTAXE CRON :

```

minute (0-59)
  |
heure (0-23)
  |
jour du mois (1-31)
  |
mois (1-12)
  |
jour de la semaine (0=dimanche)

0 6 * * 1 → Chaque lundi à 6h00
*/15 * * * * → Toutes les 15 minutes
0 0 1 * * → Le 1er de chaque mois à minuit
0 2 * * 1-5 → Du lundi au vendredi à 2h du matin

```

PARTIE V – APIs ET MISE EN PRODUCTION DES MODÈLES

5.1 EXPOSER UN MODÈLE ML VIA UNE API REST

POURQUOI UNE API ?

Un modèle en Jupyter Notebook = un outil de laboratoire.
 Un modèle derrière une API = un service consommable par n'importe quelle application.
 L'API est le contrat entre le modèle et le monde extérieur.

API AVEC FASTAPI (Standard actuel Python)

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import mlflow
import numpy as np

app = FastAPI(title="Modèle de Prédiction des Ventes", version="1.0")

# Charger le modèle depuis le registry au démarrage
model = mlflow.sklearn.load_model("models:/modele_ventes/Production")

class PredictionRequest(BaseModel):
    region: str
    mois: int
    prix_moyen: float
    nb_clients: int

class PredictionResponse(BaseModel):
    prediction: float
    confidence_interval: list[float]
    model_version: str

@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    try:
        features = preprocess(request.dict())
        prediction = model.predict([features])[0]
        ci = compute_confidence_interval(model, features)
        return PredictionResponse(
            prediction=float(prediction),
            confidence_interval=ci,
            model_version="v1.3.2"
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")

```

```

async def health_check():
    return {"status": "healthy", "model": "loaded"}

@app.get("/metrics")
async def get_metrics():
    return {"requests_total": counter, "latency_p99_ms": p99_latency}

```

BONNES PRATIQUES API ML

- Validation des inputs avec Pydantic (types, bornes, formats)
- Health endpoint (/health) pour les load balancers
- Versionnement de l'API (/v1/, /v2/)
- Rate limiting pour éviter les abus
- Authentication (API key, JWT, OAuth2)
- Logging de chaque requête (input, output, latence, erreur)
- Timeout configurable (éviter les requêtes bloquantes)
- Async pour les modèles lourds (ne pas bloquer le serveur)

5.2 CONTAINERISATION AVEC DOCKER

POURQUOI DOCKER ?

"It works on my machine" – la phrase qui tue les projets.
 Docker encapsule le code, les dépendances, et l'environnement dans un container portable et reproductible.

DOCKERFILE POUR UN MODÈLE ML

```

# Base image
FROM python:3.11-slim

# Répertoire de travail
WORKDIR /app

# Copier et installer les dépendances d'abord (cache Docker)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copier le code
COPY src/ ./src/
COPY models/ ./models/
COPY config/ ./config/

# Exposer le port
EXPOSE 8000

# Variables d'environnement
ENV MODEL_VERSION=production
ENV LOG_LEVEL=INFO

# Commande de démarrage
CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]

```

COMMANDES DOCKER ESSENTIELLES

```

# Construire l'image
docker build -t modele-ventes:v1.3 .

# Tester localement
docker run -p 8000:8000 -e API_KEY=secret modele-ventes:v1.3

# Pousser vers le registry
docker tag modele-ventes:v1.3 registry.company.com/ml/modele-ventes:v1.3
docker push registry.company.com/ml/modele-ventes:v1.3

```

DOCKER COMPOSE (environnement local complet)

```

version: '3.8'
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/mldb

```

```

- REDIS_URL=redis://cache:6379
depends_on: [db, cache]

db:
  image: postgres:15
  volumes: [postgres_data:/var/lib/postgresql/data]

cache:
  image: redis:7-alpine

mlflow:
  image: ghcr.io/mlflow/mlflow:latest
  ports: ["5000:5000"]

```

5.3 KUBERNETES – DÉPLOIEMENT ET SCALING AUTOMATIQUE

POURQUOI KUBERNETES ?

- Déployer des containers à grande échelle
- Scaling automatique selon la charge (HPA - Horizontal Pod Autoscaler)
- Self-healing : redémarrage automatique des pods en échec
- Rolling updates : zéro downtime lors des mises à jour

CONCEPTS KUBERNETES ESSENTIELS POUR ML

```

Pod          : unité de base, contient un ou plusieurs containers
Deployment   : gère le cycle de vie des pods (réplicas, rolling update)
Service      : expose les pods via un point d'accès stable (IP virtuelle)
Ingress      : routage HTTP vers les services (avec TLS)
ConfigMap    : configurations non-sensibles
Secret       : données sensibles (API keys, passwords)
HPA          : scaling automatique basé sur CPU/mémoire/métriques custom

```

DÉPLOIEMENT D'UN MODÈLE ML (deployment.yaml)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: modele-ventes
  labels:
    app: modele-ventes
    version: v1.3.2
spec:
  replicas: 3
  selector:
    matchLabels:
      app: modele-ventes
  template:
    spec:
      containers:
        - name: api
          image: registry.company.com/ml/modele-ventes:v1.3.2
          ports:
            - containerPort: 8000
          resources:
            requests:
              memory: "512Mi"
              cpu: "500m"
            limits:
              memory: "2Gi"
              cpu: "2000m"
          readinessProbe:
            httpGet:
              path: /health
              port: 8000
            initialDelaySeconds: 30
      env:
        - name: API_KEY
          valueFrom:
            secretKeyRef:
              name: ml-secrets
              key: api-key

```

STRATÉGIES DE DÉPLOIEMENT

BLUE-GREEN :

- Deux environnements identiques (blue=actuel, green=nouveau)
- Basculer le trafic d'un coup : 100% blue → 100% green
- Rollback instantané si problème
- Coût : double les ressources pendant la transition

CANARY :

- Déployer la nouvelle version à un petit % du trafic (5%, 10%, 25%...)
- Observer les métriques
- Augmenter progressivement si tout va bien
- Rollback automatique si erreurs détectées
- Idéal pour les modèles ML (A/B test en production)

SHADOW MODE :

- La nouvelle version reçoit le trafic en copie mais ne sert pas les réponses
- Comparer les prédictions shadow vs production sans impact utilisateur
- Valider le comportement avant de basculer

PARTIE VI – MONITORING ET MAINTENANCE AUTOMATISÉE

6.1 MONITORING DES MODÈLES EN PRODUCTION

POURQUOI MONITORER ?

Un modèle entraîné sur des données de 2023 ne sera pas aussi performant sur des données de 2025. Le monde change. Le modèle, lui, ne change pas automatiquement. C'est le data drift.

"Un modèle sans monitoring est une bombe à retardement."

LES 3 TYPES DE DRIFT

[A] DATA DRIFT (Covariate Shift)

La distribution des inputs X change dans le temps.

Exemple : les clients de 2025 ont des comportements différents de 2023.

Détection : comparer distribution des features en production vs données d'entraînement.

Tests : Kolmogorov-Smirnov, PSI (Population Stability Index), MMD

[B] LABEL DRIFT (Prior Probability Shift)

La distribution des labels Y change.

Exemple : le taux de fraude augmente soudainement (crise économique).

Détection : si les labels sont disponibles en prod → surveiller leur distribution.

[C] CONCEPT DRIFT

La relation entre X et Y change.

Exemple : le modèle anti-spam devient obsolète car les spammeurs s'adaptent.

C'est le plus dangereux car invisible sans les vrais labels.

Détection : surveiller les métriques de performance (accuracy, F1...) quand les labels ground truth arrivent.

MÉTRIQUES À MONITORER

MÉTRIQUES TECHNIQUES (toujours disponibles)

- Latence des prédictions (p50, p95, p99)
- Taux d'erreur de l'API (% de 5XX)
- Disponibilité (uptime %)
- Volume de requêtes
- Utilisation CPU/RAM

MÉTRIQUES ML (disponibles avec délai)

- Distribution des prédictions (score moyen, proportion par classe)
- Distribution des features d'entrée
- Performance réelle (si labels disponibles) : accuracy, F1, RMSE
- PSI (Population Stability Index) par feature

$PSI = \sum (Observed\% - Expected\%) \times \ln(Observed\% / Expected\%)$

$PSI < 0.10 \rightarrow$ pas de drift significatif

PSI $\in$ [0.10, 0.25] → drift modéré → surveiller
PSI > 0.25 → drift fort → réentraîner

STACK DE MONITORING ML

- Evidently AI : rapports de drift automatiques, dashboard HTML
- Whylogs : profiling statistique léger, intégration MLflow
- Prometheus : collecte de métriques time-series
- Grafana : dashboards et alertes visuelles sur Prometheus
- Arize AI : plateforme ML observability complète (cloud)

DÉCLENCHEMENT AUTOMATIQUE DU RÉENTRAÎNEMENT

Stratégie 1 – Schedule fixe :

- Réentraîner tous les X jours/semaines
- Simple, prévisible, mais pas adaptatif

Stratégie 2 – Trigger basé sur les métriques :

- Si PSI > 0.25 sur une feature critique → déclencher réentraînement
- Si accuracy < seuil → déclencher réentraînement + alerte
- Plus intelligent, réactif aux vraies dégradations

Stratégie 3 – Online Learning :

- Mettre à jour le modèle en continu avec chaque nouveau batch
- Adaptatif mais risque d'instabilité
- Adapté aux modèles linéaires (SGD) et certains algorithmes online

6.2 OBSERVABILITÉ COMPLÈTE – LOGS, MÉTRIQUES, TRACES

LES 3 PILIERS DE L'OBSERVABILITÉ

[1] LOGS – Enregistrer les événements

- Format structuré JSON (facilement parseable)
- Niveau : DEBUG, INFO, WARNING, ERROR, CRITICAL
- Centraliser avec : ELK Stack (Elasticsearch + Logstash + Kibana)
ou Loki + Grafana

EXEMPLE LOG STRUCTURÉ :

```
{
  "timestamp": "2025-04-18T06:23:11Z",
  "level": "INFO",
  "service": "modele-ventes",
  "version": "v1.3.2",
  "request_id": "abc-123",
  "input_features": {"region": "Nord", "mois": 4},
  "prediction": 142500.0,
  "latency_ms": 23,
  "model_version": "production-v1.3"
}
```

[2] MÉTRIQUES – Mesurer les performances

- Compteurs (nb requêtes, nb erreurs)
- Jauges (CPU actuel, connexions ouvertes)
- Histogrammes (distribution des latences)
- Exposer via /metrics (format Prometheus)

EXEMPLE AVEC PROMETHEUS CLIENT :

```
from prometheus_client import Counter, Histogram, start_http_server

REQUEST_COUNT = Counter('ml_requests_total', 'Nb de requêtes')
REQUEST_LATENCY = Histogram('ml_latency_seconds', 'Latence en secondes')

@REQUEST_LATENCY.time()
def predict(features):
    REQUEST_COUNT.inc()
    return model.predict(features)
```

[3] TRACES – Suivre les requêtes de bout en bout

- Tracer le chemin d'une requête à travers les microservices
- Identifier les goulots d'étranglement
- Outils : Jaeger, Zipkin, OpenTelemetry

=====

7.1 RPA – AUTOMATISER LES PROCESSUS MÉTIER RÉPÉTITIFS

DÉFINITION :

La RPA utilise des "robots logiciels" pour automatiser des tâches manuelles répétitives effectuées par des humains sur des interfaces graphiques ou des applications.

RPA ≠ programmation classique : le robot interagit avec l'UI comme un humain (clics, saisie, copier-coller) sans accès à l'API.

CAS D'USAGE RPA + DATA SCIENCE

[A] COLLECTE AUTOMATISÉE DE DONNÉES

- Scraper des sites web sans API disponible
- Extraire des données de portails gouvernementaux
- Agréger des rapports depuis plusieurs systèmes legacy

[B] AUTOMATISATION DE RAPPORTS

- Générer des rapports Excel/PDF à partir de données
- Envoyer des rapports par email selon un planning
- Remplir des formulaires dans des ERP anciens

[C] DÉCISION ASSISTÉE PAR ML + EXÉCUTION RPA

- ML détecte une anomalie → RPA ouvre un ticket automatiquement
- ML classe un document → RPA le dépose dans le bon dossier
- ML prédit une rupture de stock → RPA passe la commande fournisseur

PYTHON POUR LA RPA

SELENIUM / PLAYWRIGHT (automatisation web)

```
from playwright.sync_api import sync_playwright
```

```
with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://portail.exemple.com/rapports")
    page.fill("#login", "username")
    page.fill("#password", "password")
    page.click("#submit")
    page.wait_for_selector("#dashboard")
    data = page.inner_text(".tableau-ventes")
    browser.close()
```

PYAUTOGUI (automatisation desktop)

```
import pyautogui
pyautogui.click(150, 300) # Cliquer à la position (150, 300)
pyautogui.typewrite("valeur", interval=0.05)
pyautogui.hotkey('ctrl', 'c') # Copier
pyautogui.screenshot('capture.png') # Capture d'écran
```

OPENPYXL / XLWINGS (Excel automatisé)

```
import openpyxl
wb = openpyxl.load_workbook("rapport.xlsx")
ws = wb.active
ws['A1'] = "Rapport Automatisé"
ws.append([date, valeur, score])
wb.save("rapport_mis_a_jour.xlsx")
```

OUTILS RPA PROFESSIONNELS

- UiPath : leader du marché, interface visuelle, scalable
- Automation Anywhere : cloud-native, IA intégrée
- Blue Prism : entreprise, sécurisé, bancaire/assurance
- Power Automate (Microsoft) : intégré à l'écosystème Office 365

LIMITES DE LA RPA

- Fragile si l'UI change (sélecteurs cassés)
- Difficile à tester et à maintenir
- Pas adapté aux processus de haute fréquence (→ APIs)
- RPA = solution temporaire en attendant l'API ou la refonte système

8.1 LES AGENTS IA – L'AUTOMATISATION DE NIVEAU 3

DÉFINITION :

Un agent IA est un système qui perçoit son environnement, raisonne, décide et agit de façon autonome pour atteindre un objectif, en utilisant des outils disponibles et en maintenant un état.

Agent = LLM (cerveau) + Outils (bras) + Mémoire (contexte) + Objectif

ARCHITECTURE D'UN AGENT

[PERCEPTION] → Recevoir les inputs (texte, données, images, API responses)
[RAISONNEMENT] → LLM analyse la situation et planifie les actions
[ACTION] → Appeler des outils (APIs, code, bases de données, web)
[OBSERVATION] → Observer les résultats des actions
[ITÉRATION] → Répéter jusqu'à atteindre l'objectif

OUTILS DISPONIBLES POUR UN AGENT DATA SCIENCE :

- Exécuter du code Python
- Interroger une base SQL
- Rechercher sur le web
- Lire/écrire des fichiers
- Appeler des APIs externes
- Envoyer des emails ou notifications
- Accéder au feature store
- Déclencher des pipelines Airflow

CAS D'USAGE AGENTS EN DATA SCIENCE

- [A] AGENT D'ANALYSE AUTOMATIQUE
→ Reçoit : "Analyse les ventes du dernier trimestre et identifie les anomalies"
→ Actions : SQL pour extraire les données, Python pour calculer les stats, visualisation, génération du rapport texte
→ Résultat : rapport d'analyse complet, sans intervention humaine
- [B] AGENT DE MONITORING PROACTIF
→ Surveille en continu les métriques de production
→ Détecte une dégradation → analyse la cause → propose des corrections
→ Documente l'incident automatiquement
- [C] AGENT DE FEATURE ENGINEERING
→ À partir d'une description métier → génère des features pertinentes
→ Teste leur impact → sélectionne les meilleures → met à jour le feature store
- [D] AGENT DE RÉPONSE AUX INCIDENTS
→ Alerte détectée → agent analyse les logs, identifie la cause, tente une correction automatique, escalade si nécessaire

8.2 RAG – RETRIEVAL AUGMENTED GENERATION

DÉFINITION :

RAG = combiner un LLM avec une base de connaissances externe.
Au lieu de tout mémoriser dans les paramètres du modèle, le système récupère les informations pertinentes à la volée.

PIPELINE RAG

- [1] INDEXATION (offline)
Documents → Chunking → Embedding → Vector Store
- [2] RETRIEVAL (online)
Question → Embedding → Recherche similitude → Top-K chunks
- [3] GENERATION
[Question + Chunks contextuels] → LLM → Réponse

APPLICATION DATA SCIENCE AUTOMATISÉE

- Système de Q&A sur les données de l'entreprise
- Agent qui répond aux questions sur les rapports historiques
- Documentation automatique des pipelines via RAG sur le code
- Copilote d'analyse qui connaît le contexte métier de l'entreprise

OUTILS RAG

- LangChain / LlamaIndex : frameworks d'orchestration
- ChromaDB / Qdrant / Weaviate / Pinecone : vector stores
- OpenAI / Anthropic / Mistral : modèles de langage
- Sentence-Transformers : modèles d'embedding open source

8.3 AUTOMATISATION MULTI-AGENTS

CONCEPT :

Plusieurs agents spécialisés collaborent pour résoudre des problèmes complexes.
Chaque agent a un rôle précis, comme une équipe humaine.

EXEMPLE – ÉQUIPE D'AGENTS ML

- Agent Analyste → Analyse le problème, formule les hypothèses
 - Agent Ingénieur → Prépare les données, construit les features
 - Agent Data Scientist → Entraîne et évalue les modèles
 - Agent MLOps → Déploie et monitore la solution
 - Agent Reporteur → Synthétise et communique les résultats
- Orchestrateur : coordonne les agents, gère les dépendances et le flux

FRAMEWORKS MULTI-AGENTS

- AutoGen (Microsoft) : agents multi-rôles avec conversation
- CrewAI : orchestration d'équipes d'agents
- LangGraph : graphes d'agents avec état
- Swarm (OpenAI) : orchestration légère d'agents

PARTIE IX – SÉCURITÉ, GOUVERNANCE ET ÉTHIQUE DE L'AUTOMATISATION

9.1 SÉCURITÉ DES PIPELINES AUTOMATISÉS

PRINCIPES DE SÉCURITÉ FONDAMENTAUX

- [1] JAMAIS DE SECRETS EN DUR DANS LE CODE
 - Utiliser des gestionnaires de secrets : HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager
 - Variables d'environnement à la rigueur, jamais dans Git
- [2] MOINDRE PRIVILÈGE
 - Chaque composant n'a accès qu'aux données et systèmes dont il a besoin
 - Service accounts spécifiques par pipeline
 - Accès en lecture seule aux données de production si possible
- [3] CHIFFREMENT
 - Données en transit : TLS 1.3
 - Données au repos : AES-256
 - Modèles chiffrés si contiennent de l'information sensible
- [4] AUDIT TRAIL
 - Tracer chaque accès aux données, chaque prédiction, chaque modification
 - Immuabilité des logs (stockage WORM)
 - Qui a fait quoi, quand, sur quelles données
- [5] ISOLATION
 - Environnements séparés : dev / staging / production
 - Jamais de données réelles en dev sans anonymisation
 - Network policies pour isoler les composants

9.2 GOUVERNANCE DES DONNÉES ET DES MODÈLES

DATA CATALOG

- Inventaire de toutes les données disponibles
- Métadonnées : propriétaire, source, fraîcheur, qualité, RGPD
- Data lineage : tracer l'origine et les transformations de chaque donnée
- Outils : DataHub, Apache Atlas, Alation, Collibra

MODEL GOVERNANCE

- Carte du modèle (Model Card) : description, métriques, biais, limites
- Validation humaine obligatoire avant déploiement en production
- Seuils de performance minimaux obligatoires
- Processus de révision périodique (model review)
- Auditabilité : capable d'expliquer chaque décision

RGPD ET AUTOMATISATION

- Droit à l'explication : toute décision automatisée doit être explicable
- Droit à l'oubli : pouvoir supprimer les données d'un individu partout
- Minimisation des données : ne collecter que ce qui est nécessaire
- Consentement : documenter et respecter le consentement
- DPO (Data Protection Officer) : impliquer dans la conception des pipelines

9.3 ÉTHIQUE DE L'AUTOMATISATION

BIAIS ALGORITHMIQUES

- Un pipeline automatisé peut amplifier les biais présents dans les données
- Vérifier systématiquement la performance par groupe (genre, âge, ethnie)
- Métriques de fairness : parité démographique, égalité des chances
- Outils : IBM AI Fairness 360, Microsoft Fairlearn, Aequitas

RESPONSABILITÉ ET HUMAN-IN-THE-LOOP

- Toute automatisation doit avoir un humain responsable identifiable
- Décisions à fort impact (crédit, emploi, médecin) : validation humaine obligatoire
- Le système doit savoir escalader quand il n'est pas sûr
- Mécanisme de contestation des décisions automatisées

PARTIE X – CAS PRATIQUES ET ARCHITECTURES RÉELLES

10.1 CAS 1 – SYSTÈME DE RECOMMANDATION ENTIÈREMENT AUTOMATISÉ

CONTEXTE : E-commerce, recommandations produits personnalisées.

ARCHITECTURE AUTOMATISÉE COMPLÈTE

- [INGESTION – toutes les heures]
 - Kafka → events clics, achats, navigation → Bronze Layer (S3/GCS)
- [TRANSFORMATION – quotidienne, Airflow]
 - Bronze → Silver (nettoyage, déduplication)
 - Silver → Gold (matrices d'interactions, features utilisateurs)
- [FEATURE STORE – quotidienne]
 - Calcul des features : nb_achats_30j, categories_preferences, RFM_score
 - Stockage offline (Parquet) et online (Redis)
- [ENTRAÎNEMENT – hebdomadaire, Airflow + MLflow]
 - Train collaborative filtering (ALS ou SVD)
 - Évaluation : precision@10, recall@10, NDCG@10
 - Si meilleur que champion → promotion automatique
- [DÉPLOIEMENT – automatique si métriques OK]
 - Docker → Kubernetes → 3 replicas → canary 10% → 100%
- [SERVING – temps réel]
 - API FastAPI → Feature Store (online) → Modèle → Top 10 recommandations
 - Latence cible : < 50ms au p99
- [MONITORING – continu]
 - PSI sur features utilisateurs → alerte si drift
 - CTR réel des recommandations → proxy de qualité en temps réel
 - Si CTR chute de 20% → alerte → réentraînement forcé

10.2 CAS 2 – DÉTECTION DE FRAUDE EN TEMPS RÉEL

CONTEXTE : Banque, détection de transactions frauduleuses.

CONTRAINTES

- Décision en < 100ms (avant validation de la transaction)
- 99.9% de disponibilité
- Explicabilité de chaque décision (régulation)
- Zéro faux négatif acceptable (coût de la fraude >> coût du faux positif)

ARCHITECTURE

[STREAM]

Transaction → Kafka topic "transactions_brutes"

[FEATURE ENGINEERING TEMPS RÉEL] (Flink)

- Calculer en temps réel :
 - Montant vs moyenne 30j de l'utilisateur
 - Fréquence transactions dernière heure
 - Score de cohérence géographique (pays habituel ?)
 - Écart au comportement typique de l'heure

[SCORING]

- Appel API modèle (XGBoost ou LightGBM pour latence faible)
- Score de fraude [0, 1]
- SHAP values calculées pour l'explicabilité

[DÉCISION AUTOMATISÉE]

- score < 0.3 → Approuver automatiquement
- score ∈ [0.3, 0.7] → Authentification forte demandée (3D Secure)
- score > 0.7 → Bloquer + alerte client + ticket équipe fraude

[FEEDBACK LOOP]

- Labels de fraude confirmée → réentraînement hebdomadaire
- Nouvelles patterns → feature engineering manuel si besoin

[EXPLICABILITÉ AUTOMATIQUE]

- Pour chaque transaction bloquée :
 - SHAP values → Top 3 raisons générées en texte naturel
 - "Transaction bloquée car : montant 5x supérieur à votre habitude, pays inhabituel, heure atypique."

10.3 CAS 3 – RAPPORT ANALYTIQUE AUTOMATISÉ

CONTEXTE : Rapport hebdomadaire de performance envoyé automatiquement aux managers.

PIPELINE COMPLET

[LUNDI 5H – Airflow déclenche]

Task 1 – Extraction SQL

- Récupérer ventes, retours, KPIs de la semaine précédente
- Valider la fraîcheur et la complétude

Task 2 – Calcul des métriques

- CA, marge, évolution vs semaine précédente, vs objectif
- Top 10 produits, flop 10, détection d'anomalies
- Prévision semaine suivante (modèle Prophet automatique)

Task 3 – Génération des visualisations

- Graphiques Plotly exportés en PNG haute résolution
- Tableau de bord interactif Streamlit généré

Task 4 – Rédaction du commentaire (LLM)

- Prompt : "Tu es analyste business senior. Voici les KPIs : [données]. Rédige un commentaire de 3 paragraphes, en français, mettant en valeur les points clés et les actions recommandées."
- Révision automatique de la cohérence (chiffres vs texte)

Task 5 – Assemblage et envoi

- PDF généré avec template de marque

- Email envoyé à la liste de distribution (Outlook/SendGrid)
- Rapport archivé dans SharePoint / S3

[RÉSULTAT]

- À 6h le lundi, chaque manager a reçu son rapport personnalisé
- Zéro intervention humaine
- Économie : 4h analyste × 52 semaines = 208h/an économisées

SYNTHÈSE – LA MATRICE DE L'AUTOMATISATION DATA SCIENCE

DOMAINE	OUTIL CLEF	AUTOMATISE QUOI
Pipelines données	Airflow / Prefect	Extraction, transform, load
Qualité données	Great Expectations	Validation automatique
Transformation SQL	dbt	SQL versionné et testé
ML Tracking	MLflow / W&B	Expériences, modèles, métriques
Hyperparams	Optuna	Recherche des meilleurs paramètres
CI/CD ML	GitHub Actions	Tests, build, déploiement
Conteneurisation	Docker	Packaging reproductible
Orchestration	Kubernetes	Scaling, self-healing, updates
Feature Store	Feast / Tecton	Features partagées et cohérentes
Monitoring	Evidently + Grafana	Drift, latence, métriques
Streaming	Kafka + Flink	Traitement temps réel
RPA	Playwright / UiPath	Interfaces legacy, rapports
Agents IA	LangChain / CrewAI	Tâches cognitives complexes
Gouvernance	DataHub / MLflow	Lineage, catalogue, auditabilité

RÈGLE D'OR GCI :

"Automatisez ce qui est répétitif. Supervisez ce qui est critique.
Expliquez ce qui est décisif. L'excellence n'est pas dans le modèle –
elle est dans le système qui le fait vivre, s'améliorer et se défendre."

RÉFÉRENCES – ÉCOSYSTÈME PYTHON DE L'AUTOMATISATION

ORCHESTRATION

apache-airflow, prefect, dagster, luigi, celery

PIPELINES DE DONNÉES

pandas, dbt-core, great_expectations, soda-core, pandera

STREAMING

confluent-kafka, flink (PyFlink), pyspark

MLOPS

mlflow, wandb, neptune-client, dvc, bentoml, seldon-core

OPTIMISATION HYPERPARAMÈTRES

optuna, hyperopt, ray[tune], scikit-optimize

APIS ET SERVING

fastapi, uvicorn, bentoml, torchserve, triton-inference-server

CONTAINERISATION ET CLOUD

docker, kubernetes (kubectl, helm), terraform

MONITORING

evidently, whylogs, prometheus_client, opentelemetry-sdk

AGENTS IA

langchain, llama-index, autogen, crewai, langgraph

RPA

playwright, selenium, pyautogui, openpyxl, xlwings

GCI – GYKHAMINE CONCEPT INVESTIGATION

"L'automatisation n'est pas le remplacement de l'humain.
C'est la libération de l'humain pour ce que les machines ne peuvent faire :
comprendre, créer, décider avec sagesse."